

# Builder Pattern in JavaScript

## What is the Builder Pattern?

The **Builder Pattern** is a creational design pattern that separates the **construction of a complex object** from its **representation**.

In simple terms:

"Build an object step-by-step instead of creating it all at once."

Instead of having a constructor with many parameters:

```
const user = new User(  
  "John",  
  "Smith",  
  26,  
  "john@email.com",  
  "Admin",  
  true,  
  "dark"  
);
```

The Builder Pattern allows:

```
const user = new UserBuilder()  
  .setName("John")  
  .setAge(26)  
  .setEmail("john@email.com")  
  .setRole("Admin")  
  .build();
```

This makes objects:

- Easier to create.
- Easier to read.
- Easier to maintain.
- Less error-prone.

---

## Why Does the Builder Pattern Exist?

A common problem in object-oriented programming is **constructors becoming too complicated**.

Example:

```

class User {

  constructor(
    name,
    age,
    email,
    address,
    phone,
    role,
    active,
    permissions,
    theme
  ) {

    this.name = name;
    this.age = age;
    this.email = email;
    this.address = address;
    this.phone = phone;
    this.role = role;
    this.active = active;
    this.permissions = permissions;
    this.theme = theme;

  }

}

```

Creating this object is difficult:

```

const user = new User(
  "John",
  26,
  "john@email.com",
  "Ireland",
  "123456",
  "Admin",
  true,
  ["read", "write"],
  "dark"
);

```

Problems:

- Hard to remember parameter order.
- Hard to understand what each value means.
- Optional values become messy.
- Constructor becomes too large.

The Builder Pattern solves this by creating the object gradually.

---

## Basic Builder Pattern Example

### User Account Builder

Imagine creating user accounts in an application.

A user may have:

- Name
- Email
- Age
- Role
- Permissions
- Profile picture
- Settings

Not every user needs every property.

---

### Product Class

The final object we want to create:

```
class User {  
  
  constructor(builder) {  
  
    // Copy values from the builder  
    // into the final User object.  
  
    this.name = builder.name;  
    this.email = builder.email;  
    this.age = builder.age;  
    this.role = builder.role;  
    this.permissions = builder.permissions;  
  
  }  
  
  display() {  
  
    console.log(`  
      User: ${this.name}  
      Email: ${this.email}  
      Role: ${this.role}
```

```
    `);  
  
    }  
  
}
```

---

## Builder Class

The Builder controls how the object is created.

```
class UserBuilder {  
  
    constructor() {  
  
        // Default values  
  
        this.name = null;  
        this.email = null;  
        this.age = null;  
        this.role = "User";  
        this.permissions = [];  
  
    }  
  
    setName(name) {  
  
        this.name = name;  
  
        // Return this so methods can be chained  
        return this;  
  
    }  
  
    setEmail(email) {  
  
        this.email = email;  
  
        return this;  
  
    }  
  
    setAge(age) {  
  
        this.age = age;
```

```
        return this;
    }

    setRole(role) {
        this.role = role;
        return this;
    }

    addPermission(permission) {
        this.permissions.push(permission);
        return this;
    }
}

// Final step:
// Create and return the completed object.

build() {
    return new User(this);
}
}
```

---

## Using the Builder

```
const user = new UserBuilder()

    .setName("John Smith")

    .setEmail("john@example.com")

    .setAge(26)

    .setRole("Admin")

    .addPermission("read")
```

```
.addPermission("write")

.build();

user.display();
```

Output:

```
User: John Smith
Email: john@example.com
Role: Admin
```

---

## How This Works Step-by-Step

### 1. Create Builder

```
const builder = new UserBuilder();
```

The builder starts with default values:

```
{
  name: null,
  email: null,
  age: null,
  role: "User",
  permissions: []
}
```

---

### 2. Add Properties

Example:

```
builder
  .setName("John")
  .setAge(26);
```

Each method updates the builder:

Before:

```
{  
  name: null,  
  age: null  
}
```

After:

```
{  
  name: "John",  
  age: 26  
}
```

---

### 3. Build the Object

```
.build();
```

The builder creates the final object:

```
return new User(this);
```

The finished object:

```
{  
  name: "John",  
  age: 26,  
  role: "User"  
}
```

---

## Why Do Builder Methods Return `this`?

Example:

```
setName(name){  
  
  this.name = name;  
  
  return this;  
  
}
```

Returning:

```
return this;
```

allows method chaining.

Without it:

```
builder.setName("John");  
  
builder.setAge(26);
```

With it:

```
builder  
  .setName("John")  
  .setAge(26)  
  .setRole("Admin");
```

This is called:

Fluent Interface

---

## A Realistic JavaScript Example

### API Request Builder

Applications often create HTTP requests.

Without Builder:

```
const request = new Request(  
  url,  
  method,  
  headers,  
  body,  
  timeout,  
  authentication,  
  cache  
);
```

With Builder:



```
const request = new RequestBuilder()

    .setURL("/users")

    .setMethod("POST")

    .addHeader("Authorization", "Bearer token")

    .setBody({
        name: "John"
    })

    .build();
```

Implementation:

```
class Request {

    constructor(builder){

        this.url = builder.url;
        this.method = builder.method;
        this.headers = builder.headers;
        this.body = builder.body;

    }

}

class RequestBuilder {

    constructor(){

        this.url = "";
        this.method = "GET";
        this.headers = {};
        this.body = null;

    }

    setURL(url){

        this.url = url;

        return this;

    }

}
```

```
}

setMethod(method){

    this.method = method;

    return this;

}

addHeader(key, value){

    this.headers[key] = value;

    return this;

}

setBody(body){

    this.body = body;

    return this;

}

build(){

    return new Request(this);

}

}
```

---

## Common Uses of Builder Pattern in JavaScript

### 1. Creating Complex Objects

Examples:

- User profiles
- Game characters
- Configuration objects
- Documents

Problem solved:

Avoid huge constructors with many parameters.

---

## 2. API Request Creation

Examples:

- HTTP clients
- Database queries
- Search filters

Problem solved:

Create requests with optional settings.

---

## 3. UI Component Creation

Example:

```
new ButtonBuilder()  
.setText("Submit")  
.setColor("blue")  
.setSize("large")  
.build();
```

Used in:

- UI libraries
- Component frameworks

Problem solved:

Build components with many optional properties.

---

## 4. Game Development

Example:

```
new CharacterBuilder()  
.setWeapon("Sword")  
.setArmor("Shield")
```

```
.setLevel(10)
.build();
```

Problem solved:

Create many variations of complex objects.

---

## 5. Configuration Objects

Example:

```
new ServerConfigBuilder()

.setPort(3000)

.enableHTTPS()

.enableLogging()

.build();
```

Problem solved:

Avoid massive configuration constructors.

---

## Problems Builder Pattern Solves

### Problem 1: Too Many Constructor Parameters

Without Builder:

```
new Car(
    "BMW",
    "M3",
    2026,
    "Blue",
    true,
    false,
    500
);
```

Hard to understand.

Builder:

```
new CarBuilder()  
  
.setBrand("BMW")  
  
.setModel("M3")  
  
.setColor("Blue")  
  
.build();
```

Much clearer.

---

## Problem 2: Optional Properties

Without Builder:

```
new User(  
  "John",  
  undefined,  
  undefined,  
  "Admin"  
);
```

Messy.

Builder:

```
new UserBuilder()  
  
.setName("John")  
  
.setRole("Admin")  
  
.build();
```

---

## Problem 3: Object Creation Logic

Sometimes objects require preparation:

Example:

- Validation
- Default values
- Calculations

- Transformations

Builder keeps this logic separate.

---

## Builder vs Normal Class

| Normal Class                   | Builder                      |
|--------------------------------|------------------------------|
| Object created immediately     | Object created step-by-step  |
| Constructor handles everything | Builder handles construction |
| Good for simple objects        | Good for complex objects     |
| Many parameters become messy   | Fluent readable creation     |

---

## Builder vs Factory Pattern

| Builder                                   | Factory                                    |
|-------------------------------------------|--------------------------------------------|
| Creates one complex object step-by-step   | Creates different object types             |
| Focuses on construction process           | Focuses on object selection                |
| Uses methods like <code>.setName()</code> | Uses methods like <code>createCar()</code> |

Example:

Factory:

```
VehicleFactory.create("car");
```

Builder:

```
new CarBuilder()  
.setEngine("V8")  
.setColor("Red")  
.build();
```

---

# Disadvantages of Builder Pattern

## 1. More Code

A simple object:

```
const user = new User("John");
```

does not need a builder.

---

## 2. More Classes

You usually need:

- Product class
  - Builder class
  - Sometimes Director class
- 

## 3. Can Be Overused

For small objects, a builder adds unnecessary complexity.

---

# When Should You Use Builder?

Use Builder when:

- ✓ Objects have many properties
- ✓ Many properties are optional
- ✓ Object creation requires multiple steps
- ✓ Constructors are becoming difficult to maintain

Examples:

- API request builders
- Configuration objects
- Database query builders
- UI components
- Complex application models

Avoid Builder when:

- ✗ Objects only have a few properties
  - ✗ A normal constructor is already readable
  - ✗ The object does not require complex creation logic
- 

## Key Takeaway

The Builder Pattern separates:

```
Object Construction
  |
  v
Builder Class
  |
  v
Final Object
```

Instead of:

```
Huge Constructor
  |
  v
Hard-to-maintain Object Creation
```

In JavaScript, the Builder Pattern is commonly implemented using:

1. A separate Builder class.
2. Chainable methods returning `this`.
3. A final `.build()` method that creates the finished object.

It is mainly used when creating **complex objects with many optional settings** while keeping the code readable and maintainable.